BTP-1 REPORT

Quantum Computation

Name: Kshitiz Anand
Roll Number: 19EC10038
Project Guide: Dr. Sarang Pendharkar

Problem Statement:

- To study the operation of a quantum computer, the requirements for one, underlying hardware, advantages, and disadvantages of using it.
- To design a python library to simulate quantum operations to simulate qubit behaviour under the influence of quantum gates.

Introduction to quantum computing:

The first idea of a quantum computer ranges back to 1970 when the renowned physicist Richard Feynman found out that to simulate quantum physics calculations on a classical computer, it would require a lot of time, almost exponential time for a given number of atoms. In a paper published in 1982, he postulated that to simulate quantum systems, we would need to build quantum computers.

The real idea of quantum computing comes from the inherited probabilistic nature of all quantum events. So, we can not state that the qubit is surely in state 1 or state 0, there is always a probabilistic aspect to it and the state of the qubit is referred to as a superposition of state 1 and state 0.

Quantum computing is a type of computation whose operations can harness the phenomena of quantum mechanics, such as superposition, interference, and entanglement. Devices that perform quantum computations are known as quantum computers. Though current quantum computers are too small to outperform usual (classical) computers for practical applications, larger realizations are believed to be capable of solving certain computational problems, such as integer factorization (which underlies RSA encryption), substantially faster than classical computers. The study of quantum computing is a subfield of quantum information science.

Church-Turing theory:

To understand the church-turing theory we need to first understand what a turing machine is.

A Turing machine is a mathematical model of computation describing an abstract machine that manipulates symbols on a strip of tape according to a table of rules. Despite the model's simplicity, it can implement any computer algorithm.

The machine operates on an infinite memory tape divided into discrete cells, each of which can hold a single symbol drawn from a finite set of symbols called the alphabet of the machine. It has a head that, at any point in the machine's operation, is positioned over one of these cells, and a state selected from a finite set of states. At each step of its operation, the head reads the symbol in its cell. Then, based on the symbol and the machine's own present state, the machine writes a symbol into the same cell, and moves the head one step to the left or the right, or halts the computation. The choice of which replacement symbol to write and which direction to move is based on a finite table that specifies what to do for each combination of the current state and the symbol that is read.

The Church-Turing theory states that any computation problem can be solved on any computer that we hope to build in the future if and only if it can be solved on the turing machine.

A stronger quantum version of the Church-Turing theory states that ‘’a quantum turing machine can efficiently simulate any realistic model of computation’’.

The building block of quantum computers: qubits

In quantum computing, a qubit or quantum bit is a basic unit of quantum information, the quantum version of the classic binary bit physically realized with a two-state device. A qubit is a two-state (or two-level) quantum-mechanical system. Examples include the spin of the electron in which the two levels can be taken as spin up and spin down; or the polarization of a single photon in which the two states can be taken to be the vertical polarization and the horizontal polarization. In a classical system, a bit would have to be in one state or the other. However, quantum mechanics allows the qubit to be in a coherent superposition of both states simultaneously, a property that is fundamental to quantum mechanics and quantum computing. The state of the qubit evolves through the various gates operations that is done on it.

For a system of n components, a complete description of its state in classical physics requires only n bits, whereas in quantum physics it requires 2^n complex numbers.

In quantum mechanics, the general state of qubit can be represented by a linear superposition of its two orthonormal basis state vectors. These vectors are the ket 0 vector and ket 1 vector and denoted as:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Any state of the qubit can be represented as the superposition of these two basis vectors. These two orthonormal basis vectors, together called the computational basis, are said to span the two-dimensional linear vector space of qubit.

Qubit basis states can also be combined to form product basis states. A set of qubits taken together is called a quantum register. For example, two qubits could be represented in a four-dimensional linear vector space spanned by the following product basis states:

$$|00\rangle = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, |01\rangle = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, |10\rangle = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \text{and } |11\rangle = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

In general, n qubits are represented by a superposition state vector in 2^n dimensional Hilbert space.

A qubit state represented as the linear combination of the basis states as follows:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Where α and ß are complex numbers known as the probability amplitudes. When we measure the qubit in the standard basis, according to Born rule, the probability of the outcome ket 0 with value "0" is $|\alpha|^2$ and that of the outcome ket 1 with value "1" is $|ß|^2$. The sum of these probabilities comes out to be 1 as specified by the second axiom of the probability theorem.

Bloch Sphere representation of a qubit:

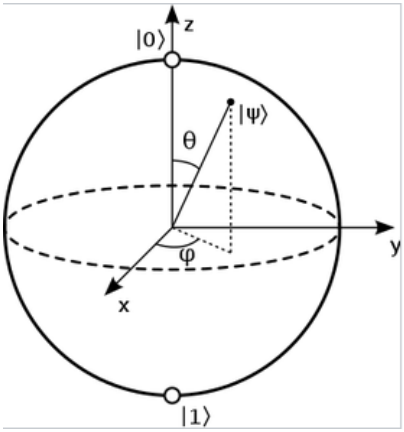
The qubit state has two degrees of freedom α and ß, which in turn have two degrees of freedom each. Due to the normalization constraint, one degree of freedom is taken away which leaves three degrees of freedom. So we need 3 different quantities to perfectly visualize a qubit state. One possible choice is that of the Hopf coordinates given as follows:

$$\alpha = e^{i\delta} \cos\frac{\theta}{2},$$
$$\beta = e^{i(\delta+\varphi)} \sin\frac{\theta}{2}.$$

For a single qubit, the global phase of the state $e^{i\delta}$ has no physical observable consequences, so we can arbitrarily choose α to be real, leaving two degrees of freedom.

$$\alpha = \cos\frac{\theta}{2},$$
$$\beta = e^{i\varphi} \sin\frac{\theta}{2},$$

Now the possible states for a single qubit can be visualized using a bloch sphere. Represented on such a 2-sphere, a classical bit could be at the “north pole” or the “south pole”, the locations where ket 0 and ket 1 are respectively. Any pure qubit state can be represented as a point on the surface of the bloch sphere. The surface of a bloch sphere is a two-dimensional space, which represents the observable state space of the pure qubit states. Block representation of qubit is shown as below:



Quantum Entanglement:

Quantum entanglement is the phenomenon that occurs when a group of particles are generated, interact, or share spatial proximity in a way such that the quantum state of each particle of the group cannot be described independently of the state of the others, including when the particles are separated by a large distance. The topic of quantum entanglement is at the heart of the disparity between classical and quantum physics: entanglement is a primary feature of quantum mechanics not present in classical mechanics.

An important distinguishing feature between qubits and classical bits is that multiple qubits can exhibit quantum entanglement. Quantum entanglement of two or more qubits allows a set of qubits to express higher correlation than is possible in classical systems. Once two qubits are entangled, it

is not possible to separate the two qubits' states and operate on them separately. Once two qubits are entangled, they will be stay entangled no matter how far are separated or how much time they are separated, they will still stay entangled and their states will be influenced by each other.

Even though a entangled system of n qubit can hold in 2^n information, when measuring it will finally collapse into one of the n states losing all the extra information that it withheld.

Operations on qubits:

The states of qubits can be altered through the use of quantum gates, analogous to how classical gates alter the state of a classical bit. The application of each quantum gate to a qubit can be represented as a matrix multiplication between the quantum gate operator and the qubit state vector. A quantum computation can be described as a network of quantum logic gates and measurements. However, any measurement can be deferred to the end of quantum computation, though this deferment may come at a computational cost, so most quantum circuits depict a network consisting only of quantum logic gates and no measurements.

The basic one qubit gates are Pauli gates, Hadamard gate, Phased gate, Virtual Z gate, Identity gate, S gate, S† gate, T gate, T† gate, U gate. Another basic operation on the qubit can be measurement.

- Pauli Gate: The Pauli gates (X, Y, Z) are the three Pauli matrices (σ_x, σ_y, σ_z) and act on a single qubit. The Pauli X, Y and Z equate, respectively, to a rotation around the x, y, and z axes of the Bloch sphere by π radians. The Pauli X gate is also referred to as the NOT gate as it completely reverses the state of the qubit state. Similarly, Pauli *Z* is sometimes called PHASE-FLIP.
- Hadamard Gate: The Hadamard Gate, commonly known as the H Gate, is a quantum gate that maps the basic states to a superposition to both the states. ket0 is mapped to (ket0 + ket1)/sqrt(2) while ket1 is mapped to (ket0 - ket1)sqrt(2). The Hadamard gate performs a rotation of π about the axis (x + z)/sqrt(2) at the Bloch sphere.
- Phase Shift Gates: The phase is a general gate that changes the phase of the qubit. The probability of measuring the basic two states remains unchanged but the overall phase of the qubit changes. It is a family of gates consisting of the phased gate (P gate), Identity Gate (I gate), S gate, T gate, S† gate, T† gate.
- U Gate: The U gates is the most general qubit gates. It rotates the qubit in the bloch sphere according to the three parameters passed to it.

Implementing the Gates in Python:

We created a python module that can efficiently simulate a quantum computer with less number of qubits. The following functionalities have been implemented:

- Initializing the Qubit:

To perform quantum gate operations on the qubit first we need to initialize the qubit. For the library the single qubit class is implemented whose object we can create by importing the library to our project space. The initialization of a single qubit system is done through the following code:

```
def __init__(self, x):
        if(x<=1 and x>=0):
            y = (1 - x**2)**0.5
            self.qubit_state = [[y],[x]]
        else:
            raise Exception("Enter a valid qubit state")
```

To create a qubit, you need to pass a variable that represents the product factor of ket1. For a qubit with state same as the classical bit 1, we need to pass 1 and for state same as classical bit 0, we need to pass 0 to the function. When the parameter passed to the function is out of the range [0,1], it can not represent a valid qubit state, so we raise an exception for the same.

Example Code:

```
import QuantumComputationLib.single_qubit as qubit

a = qubit.SingleQubit(0.95)
```

- X Gate:

The Pauli X gate is one of the most basic quantum gates. It can be considered analogous to the classical NOT gate. It changes the states ket 0 to ket1 and vice versa. The matrix operation for Pauli X gate can be represented is

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = |0\rangle\langle 1| + |1\rangle\langle 0|$$

The code implementation for Pauli X gate in the library is:

```
def xGate(self):
        self.qubit_state =
pauliGates.PauliGates.pauli_x_gate(self.qubit_state)
```

```
def pauli_x_gate(qubit_state):
        pauli_x_operator = [[0,1],[1,0]]
        output = np.dot(pauli_x_operator, qubit_state)
        return output
```

Example Code:

```
import QuantumComputationLib.single_qubit as qubit

a = qubit.SingleQubit(0.95)
a.xGate()
```

- Y Gate:

The Pauli Y gate is another one of the basic quantum gates. It rotates the qubit around the bloch sphere by π radians around the Y axis. The matrix operation for Pauli X gate can be represented is

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

$$Y = -i|0\rangle\langle 1| + i|1\rangle\langle 0|$$

The code implementation for the Pauli Y gate in the library is:

```
def yGate(self):
        self.qubit_state =
pauliGates.PauliGates.pauli_y_gate(self.qubit_state)
```

```
def pauli_y_gate(qubit_state):
        pauli_y_operator = [[0,complex(0,-1*1)],[complex(0,1),0]]
        output = np.dot(pauli_y_operator, qubit_state)
        return output
```

Example Code:

```
import QuantumComputationLib.single_qubit as qubit

a = qubit.SingleQubit(0.95)
a.yGate()
```

- Z Gate:

The Pauli Z gate rotates the qubit around the bloch sphere by π radians around the Z axis. It is also known as the PHASE-FLIP gate. The matrix operation for the Pauli Z gate can be represented as:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$Z = |0\rangle\langle 0| - |1\rangle\langle 1|$$

The code implementation for the Pauli Y gate in the library is:

```
def zGate(self):
        self.qubit_state =
pauliGates.PauliGates.pauli_z_gate(self.qubit_state)
```

```
def pauli_z_gate(qubit_state):
        pauli_z_operator = [[1,0],[0,-1]]
        output = np.dot(pauli_z_operator, qubit_state)
        return output
```

Example Code:

```
import QuantumComputationLib.single_qubit as qubit

a = qubit.SingleQubit(0.95)
a.zGate()
```

- H Gate:

The H gate, also known as the Hadamard Gate, allows us to move away from the poles of the Bloch sphere and create a superposition of |0⟩ and |1⟩. It performs a rotation of π about the axis (x + z)/sqrt(2) at the Bloch sphere. The matrix operator for the H gate can be represented as:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

The code implementation for H Gate in the library is:

```
def hGate(self):
        self.qubit_state = hGate.HadamardGate.h_gate(self.qubit_state)
```

```
def h_gate(qubit_state):
        k = 1/(2**0.5)
        h_operator = [[k,k],[k,-k]]
        output = np.dot(h_operator, qubit_state)
        return output
```

Example Code:

```
import QuantumComputationLib.single_qubit as qubit

a = qubit.SingleQubit(0.95)
a.hGate()
```

- P Gate:

The P-gate (phase gate) performs a rotation of φ around the Z-axis direction. It is a parametrised gate, it needs a number (φ) to tell it exactly what to do. The matrix operator for the P gate can be represented as:

$$P(\phi) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{bmatrix}$$

The code implementation for P Gate in the library is:

```
def pGate(self, phase):
        self.qubit_state = pGate.phased_gate.p_gate(self.qubit_state, phase)
```

```
def p_gate(qubit_state, angle):
        a = np.exp(complex(0,angle))
        p_operator = [[1,0],[0,a]]
        output = np.dot(p_operator, qubit_state)
        return output
```

Example Code:

```
import numpy as np
import QuantumComputationLib.single_qubit as qubit

a = qubit.SingleQubit(0.95)
angle = np.pi/2;
a.pGate()
```

- I Gate:

The I gate, or Identity Gate is same as the identity operation. It is same as the as a phase gate with zero degrees rotation angle. It does not change the state of the qubit but has real life importance where noise can affect the qubit system. The matrix operator for I gate can be represented as:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The code implementation for I Gate in the library is:

```
def iGate(self):
        self.qubit_state = pGate.phased_gate.i_gate(self.qubit_state)
```

```
def i_gate(qubit_state):
        i_operator = [[1,0],[0,1]]
        output = np.dot(i_operator, qubit_state)
        return output
```

Example Code:

```
import QuantumComputationLib.single_qubit as qubit

a = qubit.SingleQubit(0.95)
a.iGate()
```

- S Gate and S† Gate:

S and S† gates are a specific case of the Phase gate where the angle of rotation is pi radians and -pi radians. S and S† gates are gates that are not their own inverse. S gate is sometimes also known as the square root Z gate, as two S gate operations are same as Z gate operation. The matrix operator the S gate and the S† gate can be represented as:

$$S = \begin{bmatrix} 1 & 0 \\ 0 & e^{\frac{i\pi}{2}} \end{bmatrix}, \quad S^{\dagger} = \begin{bmatrix} 1 & 0 \\ 0 & e^{-\frac{i\pi}{2}} \end{bmatrix}$$

The code implementation for the S Gate and S† Gate in the library is:

```
def sGate(self):
        self.qubit_state = pGate.phased_gate.s_gate(self.qubit_state)

    def sDaggerGate(self):
        self.qubit_state = pGate.phased_gate.s_dagger_gate(self.qubit_state)
```

```
def s_gate(qubit_state):
        a = np.exp(complex(0, np.pi/2))
        s_operator = [[1,0],[0,a]]
        output = np.dot(s_operator, qubit_state)
        return output

def s_dagger_gate(qubit_state):
        a = np.exp(complex(0, -np.pi/2))
        s_dagger_operator = [[1,0],[0,a]]
        output = np.dot(s_dagger_operator, qubit_state)
        return output
```

Example Code:

```
import QuantumComputationLib.single_qubit as qubit

a = qubit.SingleQubit(0.95)
a.sGate()
a.sDaggerGate()
```

- T Gate and T† Gate:

T and T† gates are a specific case of the Phase gate where the angle of rotation is pi/2 radians and -pi/2 radians. Just like the S gates, T and T† gates are gates that are not their own inverse. T gate is sometimes also known as the fourth root of Z gate, as four T gate operations are same as Z gate operation. The matrix operator the T gate and the T† gate can be represented as:

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{\frac{i\pi}{4}} \end{bmatrix}, \quad T^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & e^{-\frac{i\pi}{4}} \end{bmatrix}$$

The code implementation for the T Gate and T† Gate in the library is:

```
def tGate(self):
        self.qubit_state = pGate.phased_gate.t_gate(self.qubit_state)

    def tDaggergate(self):
        self.qubit_state = pGate.phased_gate.t_dagger_gate(self.qubit_state)
```

```
def t_gate(qubit_state):
        a = np.exp(complex(0, np.pi/4))
        t_operator = [[1,0],[0,a]]
        output = np.dot(t_operator, qubit_state)
```

```
        return output

def t_dagger_gate(qubit_state):
        a = np.exp(complex(0, -np.pi/4))
        t_dagger_operator = [[1,0],[0,a]]
        output = np.dot(t_dagger_operator, qubit_state)
        return output
```

Example Code:

```
import QuantumComputationLib.single_qubit as qubit

a = qubit.SingleQubit(0.95)
a.tGate()
a.tDaggerGate()
```

- U Gate:

The U gate is the most general of all quantum gates. It can be seen as rotation in the bloch sphere by any given parameters. It takes value θ, ϕ, and λ and creates a parametrized rotation in the bloch sphere. The matrix operator for U gate can be represented as:

$$U(\theta,\phi,\lambda) = \begin{bmatrix} \cos(\frac{\theta}{2}) & -e^{i\lambda}\sin(\frac{\theta}{2}) \\ e^{i\phi}\sin(\frac{\theta}{2}) & e^{i(\phi+\lambda)}\cos(\frac{\theta}{2}) \end{bmatrix}$$

The code implementation for the U Gate in the library is:

```
def uGate(self, theta, phi, lamda):
        self.qubit_state = uGate.u_gate.u_gate(self.qubit_state, theta, phi,
lamda)
```

```
def u_gate(qubit_state, t, p, l):
        a = np.cos(t/2)
        b = -np.exp(complex(0,l))*np.sin(t/2)
        c = np.exp(complex(0,p))*np.sin(t/2)
        d = np.exp(complex(0,p+l))*np.cos(t/2)
        operator = [[a,b],[c,d]]
        output = np.dot(operator, qubit_state)
        return output
```

Example Code:

```
import numpy as np
import QuantumComputationLib.single_qubit as qubit

a = qubit.SingleQubit(0.95)
theta = np.pi/2
```

```
phi = np.pi/6
lamda = 0.5
a.uGate(theta, phi, lamda)
```

Two Qubit System:

The Quantum Computation library also provides implementation for working with systems of qubits where the qubits are entangled together. One of the basic multi qubit systems can be said as the two-qubit system. The states of the two qubits are entangled together and can be represented as a 4-row vector. The following functionalities have been implemented:

- ### Initializing the two-qubit system:

To create an object of two qubit system, firstly we would need to create two single qubits and then call the constructor for a two qubit system. The initialization of the system is implemented through the following code:

```
def __init__(self, q1, q2):
        a = q1.qubit_state[0][0]*q2.qubit_state[0][0]
        b = q1.qubit_state[0][0]*q2.qubit_state[1][0]
        c = q1.qubit_state[1][0]*q2.qubit_state[0][0]
        d = q1.qubit_state[1][0]*q2.qubit_state[1][0]
        self.qubit_state = [[a],[b],[c],[d]]
```

Example Code:

```
import QuantumComputationLib.single_qubit as qubit
import QuantumComputationLib.double_qubit_system as qubitSystem

q1 = qubit.SingleQubit(0.95)
q2 = qubit.SingleQubit(0.8)
qubits = qubitSystem.DoubleQubitSystem(q1,q2)
```

- ### CNOT Gate:

The CNOT gate, also known as the controlled NOT gate is one of the most important quantum gates. It uses one of the qubits as the control qubit and NOT operation is performed on the other qubit according to the state of the first qubit. The matrix operator for the CNOT gates can be represented as:

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

In this the first qubit is acting as the control qubit and the NOT operation is performed on the second qubit.

The code implementation for CNOT gate in the library is as follows:

```
def cnotGate(self):
        self.qubit_state = cnotGate.cnotGate.cnot_gate(self.qubit_state)
```

```
def cnot_gate(qubit_state):
        cnot_operator =  [[1,0,0,0],[0,1,0,0],[0,0,0,1],[0,0,1,0]]
        output = np.dot(cnot_operator, qubit_state)
        return output
```

Example code:

```
import QuantumComputationLib.single_qubit as qubit
import QuantumComputationLib.double_qubit_system as qubitSystem

q1 = qubit.SingleQubit(0.95)
q2 = qubit.SingleQubit(0.8)
qubits = qubitSystem.DoubleQubitSystem(q1,q2)
qubits.cnotGate()
```

- SWAP Gates:

The basic Swap gate, swap the two qubits, changes the first qubit to second and second qubit to first. It is important for computational purposes in the controlled gates where we need to change control gate accordingly. There also square root of SWAP gate and square root i SWAP gate. The matrix operations for each of the gates can be represented as:

$$SWAP = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\sqrt{\mathbf{SWAP}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{1}{2}(1+i) & \frac{1}{2}(1-i) & 0 \\ 0 & \frac{1}{2}(1-i) & \frac{1}{2}(1+i) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \pm 1/\sqrt{2} & \pm i/\sqrt{2} & 0 \\ 0 & \pm i/\sqrt{2} & \pm 1/\sqrt{2} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The code implementation for all of these gates in the library is as follows:

```
def swapGate(self):
        self.qubit_state = swapGate.swapGate.swap_gate(self.qubit_state)

    def sqrtSwapGate(self):
        self.qubit_state = swapGate.swapGate.sqrt_swap_gate(self.qubit_state)

    def sqrtISwapGate(self):
        self.qubit_state =
swapGate.swapGate.sqrt_i_swap_gate(self.qubit_state)
```

```
def swap_gate(qubit_state):
        swap_operator = [[1,0,0,0],[0,0,1,0],[0,1,0,0],[0,0,0,1]]
        output = np.dot(swap_operator, qubit_state)
        return output
```

```
    def sqrt_swap_gate(qubit_state):
        sqrt_swap_operator = [[1,0,0,0],[0,complex(0.5,0.5),complex(0.5,-
0.5),0],[0,complex(0.5,-0.5),complex(0.5,0.5),0],[0,0,0,1]]
        output = np.dot(sqrt_swap_operator, qubit_state)
        return output

    def sqrt_i_swap_gate(qubit_state):
        a = 1/np.sqrt(2)
        sqrt_i_swap_operator =
[[1,0,0,0],[0,a,complex(0,a),0],[0,complex(0,a),a,0],[0,0,0,1]]
        output = np.dot(sqrt_i_swap_operator, qubit_state)
        return output
```

Code Examples:

```
import QuantumComputationLib.single_qubit as qubit
import QuantumComputationLib.double_qubit_system as qubitSystem

q1 = qubit.SingleQubit(0.95)
q2 = qubit.SingleQubit(0.8)
qubits = qubitSystem.DoubleQubitSystem(q1,q2)
qubits.swapGate()
qubits.sqrtSwapGate()
qubits.sqrtISwapGate()
```

- Sycamore Gate:

The sycamore is one of the most used realistic gates. It does both a partial swap and controlled phase rotation of the $|11\rangle$ state. The matrix operation for the sycamore gate can be represented as:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -i & 0 \\ 0 & -i & 0 & 0 \\ 0 & 0 & 0 & e^{(-i\pi/6)} \end{bmatrix}$$

The code implementation for the gate in the library is as follows:

```
def sycamoreGate(self):
        self.qubit_state =
sycamoreGate.sycamoreGate.sycamore_gate(self.qubit_state)
```

```
def sycamore_gate(qubit_state):
        a = np.exp(complex(0,-np.pi/6))
        sycamore_operator = [[1,0,0,0],[0,0,complex(0,-1),0],[0,complex(0,-
1),0,0],[0,0,0,a]]
        output = np.dot(sycamore_operator, qubit_state)
        return output
```

Example Code:

```
import QuantumComputationLib.single_qubit as qubit
import QuantumComputationLib.double_qubit_system as qubitSystem

q1 = qubit.SingleQubit(0.95)
q2 = qubit.SingleQubit(0.8)
qubits = qubitSystem.DoubleQubitSystem(q1,q2)
qubits.sycamoreGate()
```

- CZ Gate:

The CZ gates is one of the quantum gates in development. It is Controlled Z gate, where the first qubit acts as the control qubit. The matrix operator for the gate can be represented as:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$$

The code implementation for CZ gate in the library is as follows:

```
def czGate(self):
        self.qubit_state = czGate.czGate.cz_gate(self.qubit_state)
```

```
def cz_gate(qubit_state):
        cz_operator = [[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,-1]]
        output = np.dot(cz_operator, qubit_state)
        return output
```

Simulating an actual circuit through the library:

We will be simulating a basic quantum circuit with two qubits on which we will perform certain gate operations separately, then entangle their states, then perform CNOT with a swap then end with a CZ gate operation.

```
import numpy as np
import QuantumComputationLib.single_qubit as qubit
import QuantumComputationLib.double_qubit_system as qubitSystem

q1 = qubit.SingleQubit(0.95)
q1.xGate()
q1.pGate(np.pi/3)
q1.hGate()

q2 = qubit.SingleQubit(0.8)
q2.yGate()
q2.sDaggerGate()
q2.hGate()

qubits = qubitSystem.DoubleQubitSystem(q1,q2)
qubits.cnotGate()
```

```
qubits.swapGate()
qubits.czGate()
qubits.print_state()
```

The output of the operation comes out to be:

((0.4400040232519076-0.3613250762860442j))|00} + ((-0.34632905327593155-0.23642511631807617j))|01} + ((-0.2236709467240682-0.5235748836819236j))|10} + ((-0.1299959767480922+0.39867492371395563j))|11}

Hence the library can simulate basic quantum circuits accurately with ease.

Future Prospects:

- The currently available qubits that we have are affected by noise which leads the state of qubits to vary with time and collapse finally into ket0 state. We can also implement this functionality into the library where we leave the qubit for a short amount of time and the state of the qubit changes accordingly.
- The currently available qubits are only available for 10^{-6} second and they lose their property after that. We can implement a timer along with our library that calculates the total time that the complete circuit simulation would take and then we can check the feasibility of the computation accordingly.
- We can include more qubits in a system.
- We can publish the library as a package to PyPi where it can be used and contributed to by anyone around the world.

Repository Link for the Library:

https://github.com/Kshitiz9806/QuantumComputationLib.git

References:

- Google Cirq Gates: https://quantumai.google/cirq/build/gates
- Qiskit Gates: https://qiskit.org/textbook/ch-states/single-qubit-gates.html
- Google Cirq Datasheet: https://quantumai.google/hardware/datasheet/weber.pdf
- https://cdn.inst-fs-iad-prod.inscloudgate.net/6fbee6d9-a440-4f40-8e69-eb93605e7787/Quantum%20gate%20-%20Wikipedia.pdf?token=eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCIsImtpZCI6ImNkbiJ9.eyJyZXNvdXJjZSI6Ii82ZmJlZTZkOS1hNDQwLTRmNDAtOGU2OS1lYjkzNjA1ZTc3ODcvUXVhbnR1bSUyMGdhdGUlMjAtJTIwV2lraXBlZGlhLnBkZiIsInRlbmFudCI6ImNhbnZhcyIsInVzZXJfaWQiOm51bGwsImlhdCI6MTY2OTI0MTQzNSwiZXhwIjoxNjY5MzI3ODM1fQ.4DLAesRJMlE_RWvP0pcfxq0xGponPRa6NfNLvorEu2IEu9MjX47t0FvmaIpMy0k8TENn112KkxowbqrCTjcXbg&download=1&content_type=application%2Fpdf